Final Project - Titanic Survival Studies

Kaiping Liu, Nick Zhu
DS 100

Table of Content

1. Dataset Overview
2. Variable Relationship
3. Statistical Test
4. Causal Interpretation
5. Logistic Regression Model

Dataset Overview

```
df.head(10)
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

Variable	Definition	Key
survival	Survival	0 = No, 1 = Yes
pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
sex	Sex	
Age	Age in years	
sibsp	# of siblings / spouses aboard the Titanic	
parch	# of parents / children aboard the Titanic	
ticket	Ticket number	
fare	Passenger fare	
cabin	Cabin number	
embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

Dataset - Missing Data and Cleaning

```python
# Display basic information about the dataset
print("Dataset Info:")
df.info()
```

✓ 0.0s

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1309 entries, 0 to 1308
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  1309 non-null   int64
 1   Survived     1309 non-null   int64
 2   Pclass       1309 non-null   int64
 3   Name         1309 non-null   object
 4   Sex          1309 non-null   object
 5   Age          1046 non-null   float64
 6   SibSp        1309 non-null   int64
 7   Parch        1309 non-null   int64
 8   Ticket       1309 non-null   object
 9   Fare         1308 non-null   float64
 10  Cabin        295 non-null    object
 11  Embarked     1307 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 122.8+ KB
```

Age: 177 missing values (19.9% of data)

Impute missing **Age** values using median age grouped by Pclass and Sex

Fare: 1 missing value (~0.00% of data)

Impute missing **Fare** value using median fare grouped by Pclass

Cabin: 687 missing values (77.1% of data)

Drop the **Cabin** column due to excessive missing data

Embarked: 2 missing values (0.2% of data)

Replaced with the mode value

Dataset - Missing Data and Cleaning

```python
# Display basic information about the dataset
print("Dataset Info:")
df.info()
```

✓ 0.0s

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1309 entries, 0 to 1308
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  1309 non-null   int64
 1   Survived     1309 non-null   int64
 2   Pclass       1309 non-null   int64
 3   Name         1309 non-null   object
 4   Sex          1309 non-null   object
 5   Age          1046 non-null   float64
 6   SibSp        1309 non-null   int64
 7   Parch        1309 non-null   int64
 8   Ticket       1309 non-null   object
 9   Fare         1308 non-null   float64
 10  Cabin        295 non-null    object
 11  Embarked     1307 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 122.8+ KB
```

For categorical data:

Sex: object → int

Male = 1, Female = 0

Embarked: object “C”, “Q”, “S”→ boolean → int

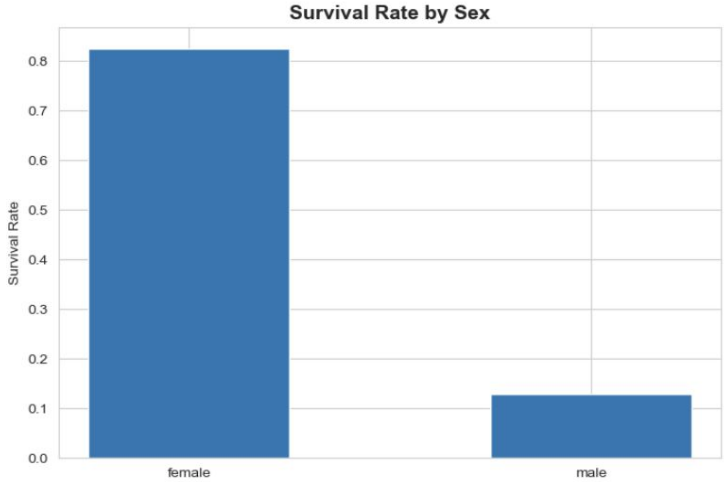
Embarked is converted into three dummy variables (C, Q, S), each encoded as 0/1

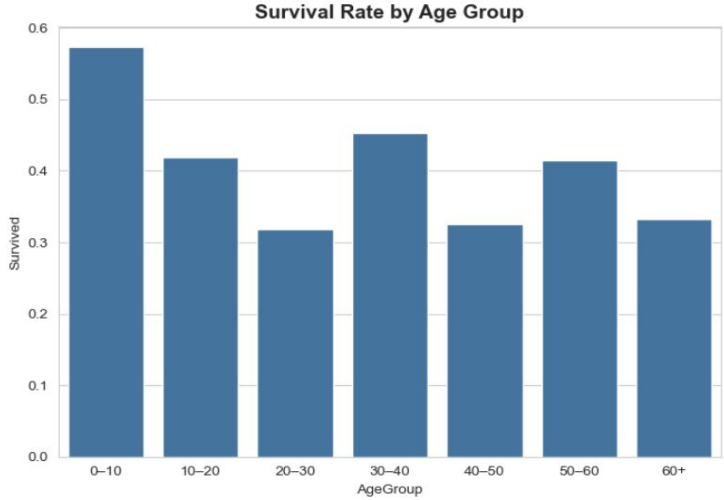
Dataset Overview

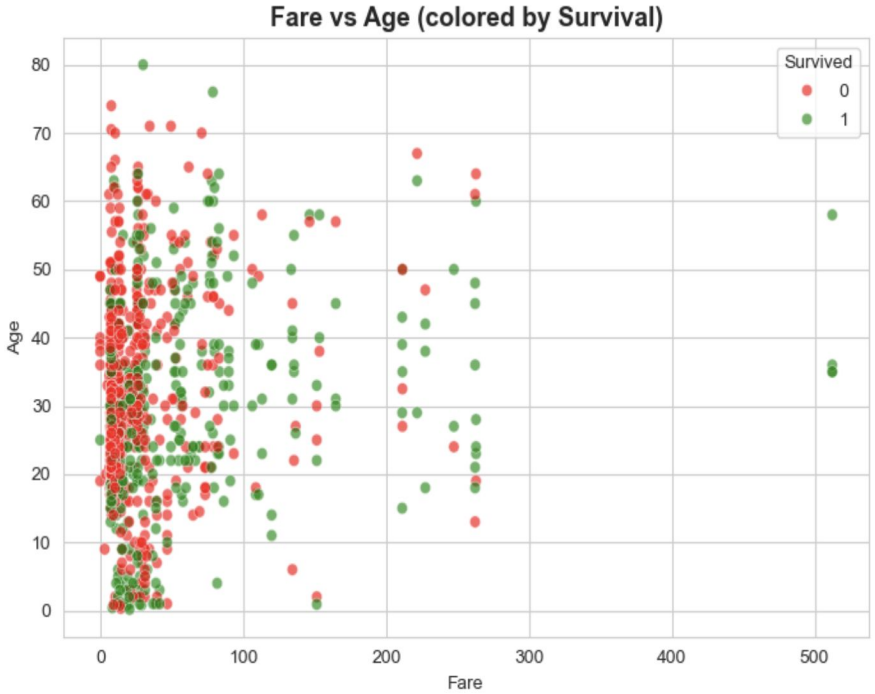
```
df_clean.head()
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	Sex_num	C	Q	S
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S	1	0	0	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	0	1	0	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S	0	0	0	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S	0	0	0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S	1	0	0	1


Survival Rate by Sex
Survival Rate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
female
male


Survival Rate by Age Group
Survived
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0–10
10–20
20–30
30–40
40–50
50–60
60+
AgeGroup


Fare vs Age (colored by Survival)
Survived
0
1
Age
0
10
20
30
40
50
60
70
80
Fare
0
100
200
300
400
500

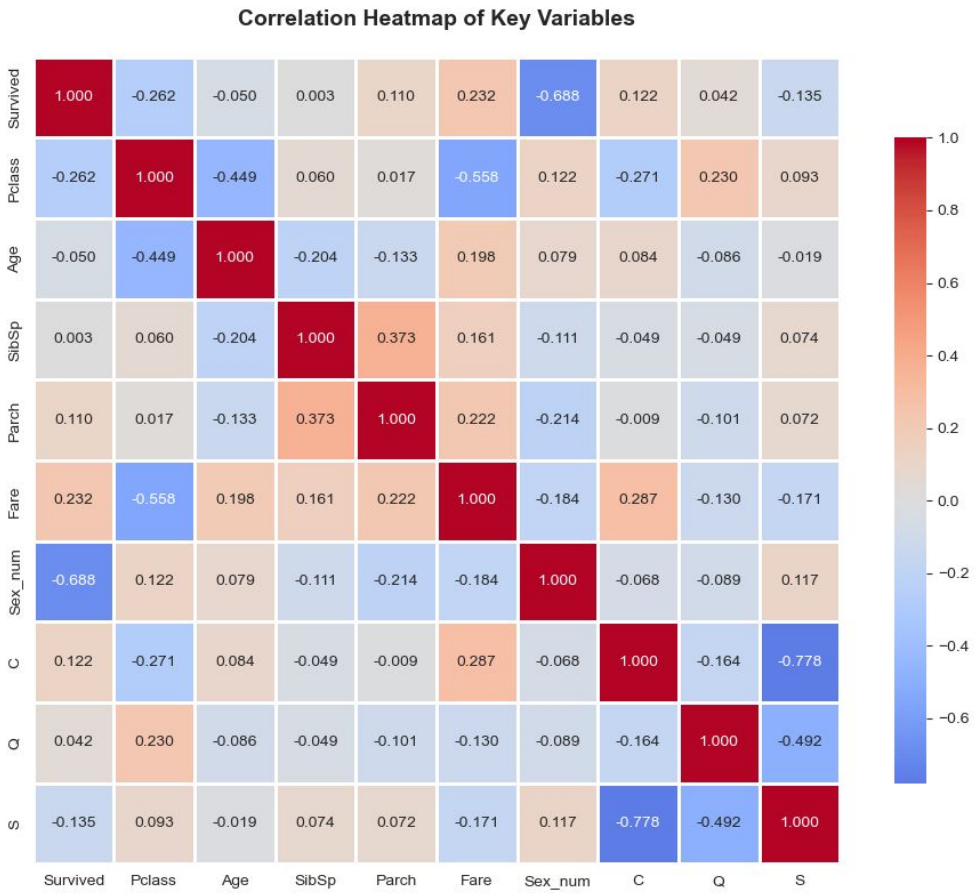
Variable Relationship

The heatmap highlights several meaningful relationships among key variables in the dataset:

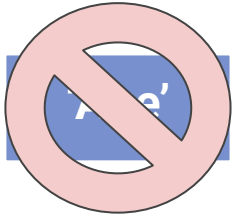
- **Strong, clear predictors:**
  - **Sex_num** shows the strongest association with survival (corr ≈ −0.688)
  - **Fare** (corr ≈ 0.232) and **Pclass** (corr ≈ −0.262)

- **Ambiguous or weaker relationships:**
  - **Age**
  - **Parch and Sibsp**
  - **Embarked (C,Q,S)**

- Strong correlations among the dummy variables (C,Q,S) reflect that a passenger can only belong to one port category.
- **SibSp** and **Parch** show mixed signals: they correlate with each other but weakly with survival, suggesting family size influences survival in complex ways.



Statistical Test



H_0: The mean age of survivors **equals** the mean SibSp of non-survivors.

H_1: The mean age of survivors **differs** from the mean SibSp of non-survivors.

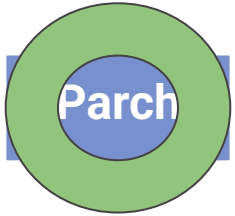
```python
# Two-sample t-test: Age of survivors vs. non-survivors
survivors_age = df_clean[df_clean['Survived'] == 1]['Age']
non_survivors_age = df_clean[df_clean['Survived'] == 0]['Age']

# Perform two-sample t-test
t_stat, p_value = stats.ttest_ind(survivors_age, non_survivors_age)

print("Two-Sample T-Test: Age of Survivors vs. Non-Survivors")
print("=" * 60)
print(f"Survivors - Mean age: {survivors_age.mean():.2f}, Std: {survivors_age.std():.2f}, n={len(survivors_age)}")
print(f"Non-survivors - Mean age: {non_survivors_age.mean():.2f}, Std: {non_survivors_age.std():.2f}, n={len(non_survivors_age)}")
print(f"\nT-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")
```

✓ 0.0s

```
Two-Sample T-Test: Age of Survivors vs. Non-Survivors
============================================================
Survivors - Mean age: 28.49, Std: 14.14, n=494
Non-survivors - Mean age: 29.73, Std: 12.61, n=815

T-statistic: -1.6527
P-value: 0.0986
```

At a significance level of $\alpha=0.05$, we fail to reject H_0. There is no statistically significant difference in mean age between survivors and non-survivors in this dataset.

Statistical Test



H_0: The mean Parch of survivors **equals** the mean Parch of non-survivors.

H_1: The mean Parch of survivors **differs** from the mean Parch of non-survivors.

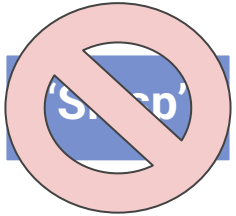
```python
# Two-sample t-test: Parch of survivors vs. non-survivors
survivors_parch = df_clean[df_clean['Survived'] == 1]['Parch']
non_survivors_parch = df_clean[df_clean['Survived'] == 0]['Parch']

# Perform two-sample t-test
t_stat_parch, p_value_parch = stats.ttest_ind(survivors_parch, non_survivors_parch)

print("Two-Sample T-Test: Parch of Survivors vs. Non-Survivors")
print("=" * 60)
print(f"Survivors - Mean Parch: {survivors_parch.mean():.4f}, Std: {survivors_parch.std():.4f}, n={len(survivors_parch)}")
print(f"Non-survivors - Mean Parch: {non_survivors_parch.mean():.4f}, Std: {non_survivors_parch.std():.4f}, n={len(non_survivors_parch)}")
print(f"T-statistic: {t_stat_parch:.4f}")
print(f"P-value: {p_value_parch:.4f}")
```

✓ 0.0s

```
Two-Sample T-Test: Parch of Survivors vs. Non-Survivors
============================================================
Survivors - Mean Parch: 0.5061, Std: 0.8888, n=494
Non-survivors - Mean Parch: 0.3117, Std: 0.8433, n=815
T-statistic: 3.9613
P-value: 0.0001
```

At a significance level of $\alpha=0.05$, we **reject** H_0 and conclude that traveling with parents/children is associated with higher survival odds.

Statistical Test



H_0: The mean SibSp of survivors **equals** the mean SibSp of non-survivors.

H_1: The mean SibSp of survivors **differs** from the mean SibSp of non-survivors.

```python
# Two-sample t-test: SibSp of survivors vs. non-survivors
survivors_sibsp = df_clean[df_clean['Survived'] == 1]['SibSp']
non_survivors_sibsp = df_clean[df_clean['Survived'] == 0]['SibSp']

# Perform two-sample t-test
t_stat_sib, p_value_sib = stats.ttest_ind(survivors_sibsp, non_survivors_sibsp)

print("Two-Sample T-Test: SibSp of Survivors vs. Non-Survivors")
print("=" * 60)
print(f"Survivors - Mean SibSp: {survivors_sibsp.mean():.4f}, Std: {survivors_sibsp.std():.4f}, n={len(survivors_sibsp)}")
print(f"Non-survivors - Mean SibSp: {non_survivors_sibsp.mean():.4f}, Std: {non_survivors_sibsp.std():.4f}, n={len(non_survivors_sibsp)}")
print(f"\nT-statistic: {t_stat_sib:.4f}")
print(f"P-value: {p_value_sib:.4f}")
```

✓ 0.0s

```
Two-Sample T-Test: SibSp of Survivors vs. Non-Survivors
============================================================
Survivors - Mean SibSp: 0.5020, Std: 0.8000, n=494
Non-survivors - Mean SibSp: 0.4969, Std: 1.1645, n=815

T-statistic: 0.0857
P-value: 0.9317
```

The p-value is approximately 0.93, which is far greater than 0.05. We fail to reject H_0. This suggests that there is no statistically significant difference in the average number of siblings/spouses between those who survived and those who did not.

Causal Interpretation

Selection Bias	Confounding Variables	Correlation ≠ Causation
Passengers differed systematically in wealth, cabin location, and travel companions. Therefore, differences in survival across Pclass and Fare reflect pre-existing differences, not the causal effect of class.	Sex, Age, Pclass, and Fare are intertwined with social norms (eg. “Women and children first" policy), group travel behavior, and ship layout. These confounders affect both the “treatment” variables and survival, making correlations non-causal.	Strong associations (Sex, Pclass) cannot be interpreted causally, and weak correlations (Age) do not imply no causal effect. The data is observational, so correlation only describes patterns, not mechanisms.

Predictive Model - Logistic Regression

1. Prepare Features:

```python
# Select features for the model
features_to_include = ['Pclass', 'Parch', 'Fare', 'Sex_num', 'C', 'S']
X = df_clean[features_to_include]
y = df_clean['Survived']

print("Features used:", X.columns.tolist())
print(f"Shape: {X.shape}")
```

✓ 0.0s

```
Features used: ['Pclass', 'Parch', 'Fare', 'Sex_num', 'C', 'S']
Shape: (1309, 6)
```

2. Train-Test Split: 80% - 20%
3. Train Logistic Regression Model:

```python
lr_model = LogisticRegression(penalty='l2', C=1.0, max_iter=1000, random_state=42)
lr_model.fit(X_train, y_train)

print("Logistic Regression Model Trained")
print("=" * 50)
print(f"Model parameters: penalty='l2', C=1.0, max_iter=1000")
print(f"Number of iterations: {lr_model.n_iter_[0]}")
```

✓ 0.0s

```
Logistic Regression Model Trained
==================================================
Model parameters: penalty='l2', C=1.0, max_iter=1000
Number of iterations: 75
```

Predictive Model - Logistic Regression

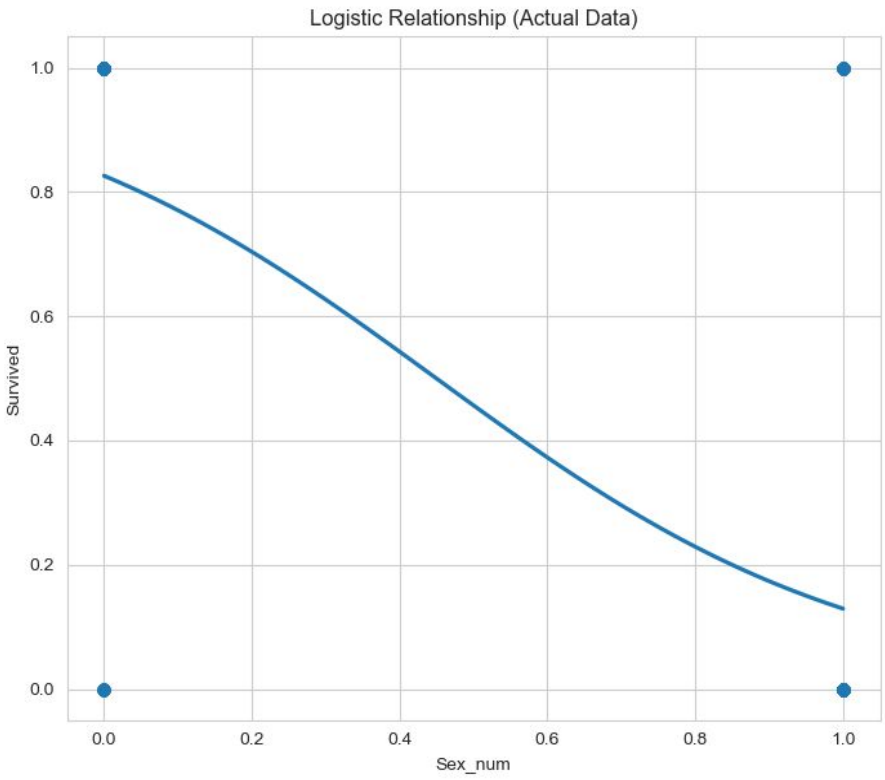
```python
# Display model coefficients
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_[0]
})
coefficients = coefficients.sort_values('Coefficient', ascending=False)

print("Logistic Regression Coefficients:")
print("=" * 50)
print(coefficients.to_string(index=False))
```

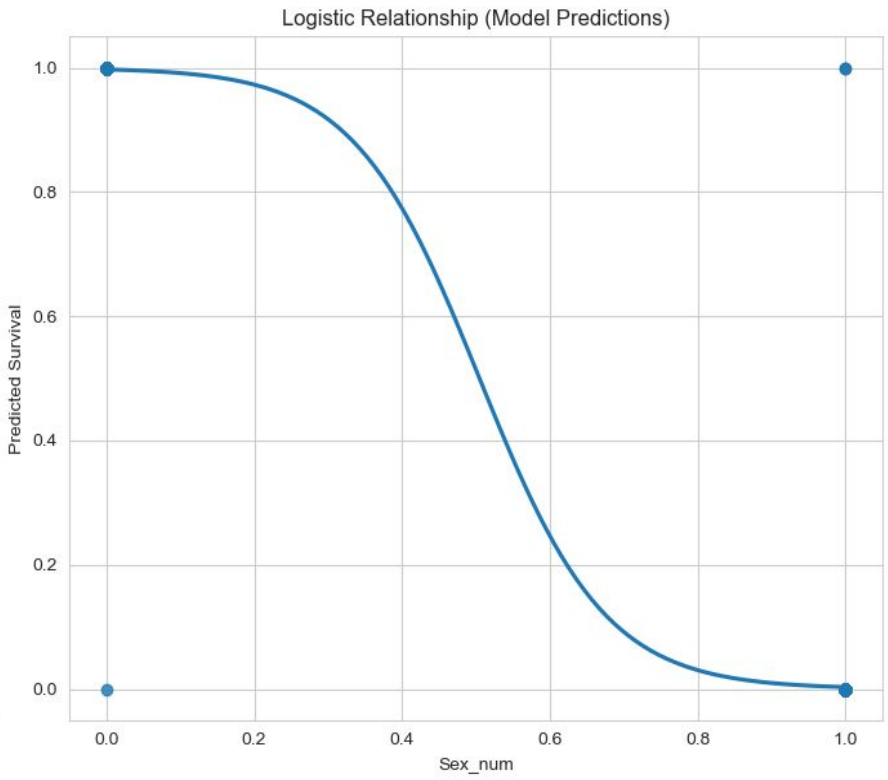
✓ 0.0s

```
Logistic Regression Coefficients:
==================================================
Feature  Coefficient
   Fare     0.003340
  Parch    -0.172805
      S    -0.261205
      C    -0.303179
 Pclass    -0.724112
Sex_num    -3.555892
```

- **Sex_num** has the largest negative coefficient, meaning the trained model learns that being male drastically reduces the predicted odds of survival.
- **Pclass** has a strong negative effect, indicating the model identifies lower-class passengers as significantly less likely to survive.
- **Parch, Embarked indicators (C,S)** and **Fare** contribute weakly

Predictive Model





Predictive Model - Performance

```
In-Sample Performance (Training Set):
==================================================
Accuracy: 0.8625 (86.25%)

Classification Report:
--------------------------------------------------
                 precision    recall  f1-score   support

Did Not Survive       0.88      0.91      0.89       652
       Survived       0.84      0.79      0.81       395

       accuracy                           0.86      1047
      macro avg       0.86      0.85      0.85      1047
   weighted avg       0.86      0.86      0.86      1047


Confusion Matrix:
--------------------------------------------------
[[592  60]
 [ 84 311]]

True Negatives: 592, False Positives: 60
False Negatives: 84, True Positives: 311
```

```
Out-of-Sample Performance (Test Set):
==================================================
Accuracy: 0.8435 (84.35%)

Classification Report:
--------------------------------------------------
                 precision    recall  f1-score   support

Did Not Survive       0.85      0.91      0.88       163
       Survived       0.83      0.74      0.78        99

       accuracy                           0.84       262
      macro avg       0.84      0.82      0.83       262
   weighted avg       0.84      0.84      0.84       262


Confusion Matrix:
--------------------------------------------------
[[148  15]
 [ 26  73]]

True Negatives: 148, False Positives: 15
False Negatives: 26, True Positives: 73
```

Thank you